

Föreläsning 1

DD1385

- Kursöversikt
- Javaintroduktion

Kursmål

- Använda objektorienterade tekniker vid eget programmeringsarbete
- Redogöra för och tillämpa kriterier för god objektorienterad design
- Redogöra för de vanligaste designmönstren inom objektorienterad programutveckling samt välja lämpliga mönster för enkla tillämpningsexempel
- Använda UML-klassdiagram för att på ett överskådligt och tydligt sätt planera och dokumentera eget programmeringsarbete
- Läs och förstå UML-klassdiagram, t.ex som introduktion till nya designmönster
- Använda Javas biblioteksklasser och ramverk

Objektorienterad programmering i Java

- Klasser, objekt, metod, konstruktor, arv, static
- Abstrakta klasser, interface
- Klassbibliotek och ramverk
- Designmönster i Java-biblioteken
- Grafiska gränssnitt, i första hand med Swing
 - Enkla komponenter, t.ex. JLabel, JButton
 - Avancerade komponenter, t.ex. JList, JTree, JEditorPane och Parallella processer: trådar
- Klient-Server

Tips till pythonprogramerare

- Lär dig OOP ordentligt
 - Klass, objekt, konstruktor
 - Interface, abstrakt klass
 - Arv, composition
 - Encapsling
 - Ansvarsområde
 - Nyckelorden: `public`, `private`, `protected`
- Tänk mer på datatyper
- Utnyttja `record` men först när du har lärt dig grunderna
- I java finns konstanter på riktigt använd det: `final static`
- Använd dig mer av List/ArrayList, Map och Set
- Använd generiska typer

Java-begrepp att ta upp idag

- Kompilator, Java Bytecode, JVM
- Klass, Konstruktör, objekt
- Styrstrukturer i java
- Instansvariabel, instansmetod
- Klassvariabel, klassmetod
- Typer, typdeklarationer
- Arv, konstruktör vid arv
- `public static void main(String[] args)`

Kompilator

Kompilering av Java-kod

- Javakällkod måste kompileras innan den kan exekveras.
- Med kommandot **javac** kompileras Java-källkod.

Kompilatorns uppgifter:

1. **Kontrollera koden** – kompilatorn kontrollerar att koden följer Javas syntax och regler. Vid fel visas ett felmeddelande.
2. **Översätta koden** – om inga fel hittas översätts Java-koden till **Java bytecode**.
 - Vid en lyckad kompilering skapas en eller flera filer med filändelsen **.class**.
 - Med kommandot **java** körs Java-programmets bytecode.

Flödet:

javac Javakällkod → *klassnamn.class* fil → **java** *klassnamn* → körning

Exempel

```
class Hej{  
    public static void main(String[] args){  
        System.out.println("Java är enkelt!");  
    }  
}
```

```
}
```

```
> ls
```

```
Hej.java.
```

```
> javac Hej.java
```

```
> ls
```

```
Hej.java Hej.class
```

```
> java Hej
```

```
Java är enkelt!
```

← finns endast en fil

← kompileras java filen

← Här ser vi att Hej.class har skapats

← Här kommer java exekvera Hej.class

Datatyper

Primitiva datatyper	Referensdatatyper
byte short int long double float char boolean	String Scanner System Integer Double Boolean ...

Automatisk typomvandling

- byte 8 bitar
- short, char 16 bitar
- int, float 32 bitar
- long, double 64 bitar

Statisk och dynamisk typning

Python är ett språk med dynamisk typning medan för java gäller statisk typning.

Python	Java
age = 20 age = 23 age = 20.5	int age = 20; age = 23; age = 20.5; //ger fel

Metoder

Python	Java
<pre>def myndig(alder): if alder>18: return True return False</pre>	<pre>boolean myndig(int alder){ if (alder >18) return true; return false; }</pre>

Styrstrukturer

Python	<pre>counter = 1 while counter < 100 : print(100-counter, "kvar!") counter += 1</pre>
Java	<pre>int counter = 1; while (counter < 100){ System.out.println((100-counter)+" kvar!"); counter += 1; }</pre>

Styrstrukturer

Python	<pre>kortLista = ["en", "kort", "lista"] for k in kortLista: print(k)</pre>
Java	<pre>List<String> kortLista = Arrays.asList("en", "kort", "lista"); for (String k : kortLista) System.out.println(k);</pre>

Styrstrukturer i Java utan direkt motsvarighet i Python

```
int counter=1;
do {
    System.out.println(counter);
    counter+=1;
} while (counter <100) ;
```

```
for(int counter = 1; counter < 100; counter++){
    System.out.println(counter);
}
```

Array

- En array är som en lista med statisk typning och fast storlek.

```
String[] enLista = new String[4];  
enLista[0] = "Kusin vitamin";  
enLista[1] = "Krakel spektakel";  
enLista[2] = "Opsis kallopsis";  
enLista[3] = "Fröken Hit o dit";
```

Arrayens storlek

- Annat sätt att skapa arrayer:

```
String[] enLista = {"Kusin vitamin",  
                    "Krakel spektakel",  
                    "Opsis kallopsis"};
```

```
int[] enAnnanLista = new int[4];  
int antalElement = enAnnanLista.length;
```


Klass och konstruktor

Python	<pre>class Person: def __init__(self,n,a): self.name = n self.age = a</pre>
Java	<pre>class Person{ String name; int age; Person(String n, int age){ name = n; this.age = age; } }</pre> Deklaration av instansvariabler Konstruktorn har samma namn som klassen och ingen returtyp

Instansering / skapa objekt

Python	<pre>P = Person("kalle", 24)</pre>
Java	<pre>Person p = new Person("kalle",24);</pre> eller <pre>Person p; p = new Person("kalle",24);</pre>

Instansmetod

Python	Java
<pre>class Person: ... def getAge(self): return self.age def setAge(self, nAge): self.age = nAge</pre>	<pre>class Person{ String name; int age; ... int getAge() { return age; } void setAge(int nAge) { age = nAge; } }</pre>

Metodanrop

Python	<pre>p = Person("kalle", 24) age = p.getAge();</pre>
Java	<pre>Person p = new Person("kalle", 24); int age = p.getAge();</pre>

Klassvariabel (ska användas sparsam)

Python	<pre>class PhusSkylt: ledigaPlatser = 300 def __init__(self, reklam): self.reklam = reklam</pre>
Java	<pre>class PhusSkylt{ String reklam; static int ledigsaPlatser = 300; PhusSkylt (String rekl){ reklam=rekl; } }</pre>

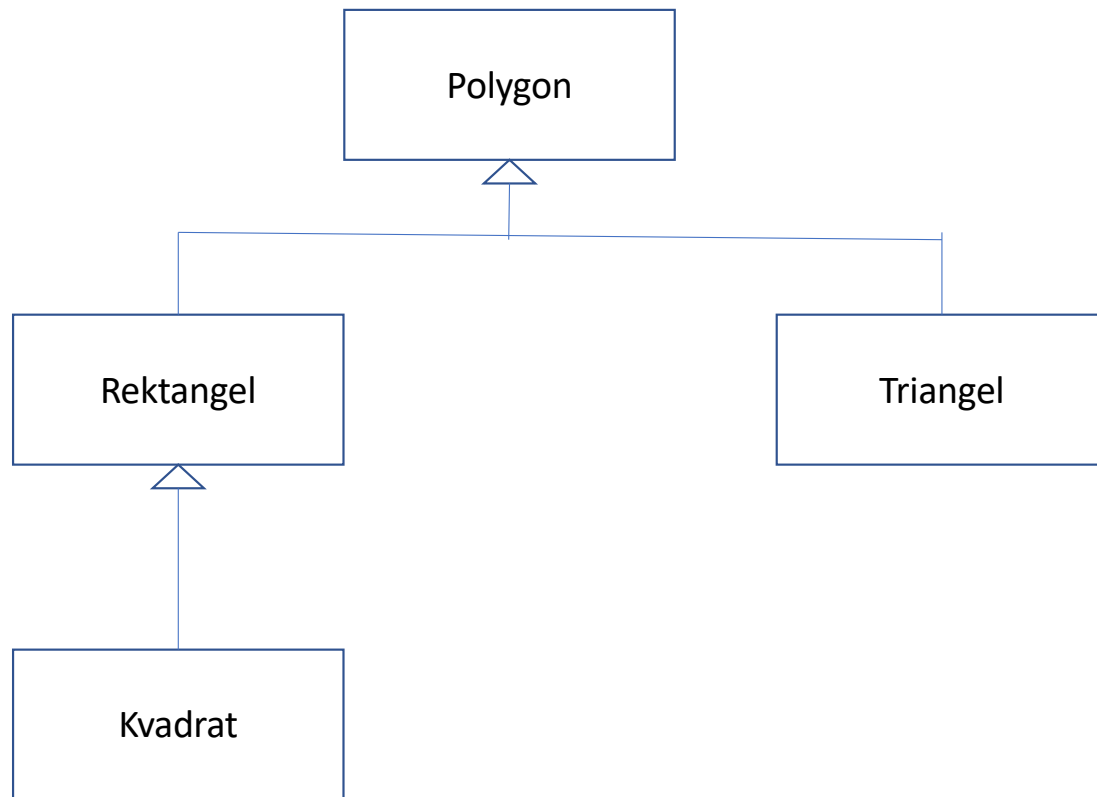
Klassmetod (ska användas sparsam)

Python	<pre>class Person: ... def getAmount(): return Person.amount staticmethod(getAmount)</pre>
Java	<pre>class Person{ ... static int getAmount(){ return Person.amount; } }</pre>

Arv

Python	<pre>class Bil(Fordon): def __init__(self, antalHjul): ...</pre>
Java	<pre>class Bil extends Fordon{ ... Bil (int antalHjul) this.antalHjul = antalHjul; } }</pre>

Arv exempel



Hur ser koden för Triangel ut?

```
class Polygon{
    int[] kanter;
    Polygon(int... k){
        kanter=new int[k.length];
        int j=0;
        for (int i : k) {
            kanter[j]=i;
            j++;
        }
    }
    int omkrets(){
        int omkrts=0;
        for (int i=0;i<kanter.length; i++)
            omkrts+=kanter[i];
        return omkrts;
    }
}
```

```
class Rektangel extends Polygon{
    Rektangel(int k1, int k2){
        super(k1,k2,k1,k2);
    }
    int area(){
        return kanter[0]*kanter[1];
    }
}
```

```
class Kvadrat extends Rektangel{
    Kvadrat(int k){
        super(k,k);
    }
}
```

Modifierare

- `public:` åtkomlig från alla andra klasser
- `private:` åtkomlig endast från samma klass
- `protected:` åtkomlig i paketet och subklasser
- **Paketprivat:** åtkomlig i paketet